R1.01 - Initiation au Développement : Variables, Modèle Mémoire RAM, Types Primitifs et Structures Conditionnelles

Thème principal : Variables, Allocation Mémoire, Typage, Overflow et Structures Conditionnelles

Synthèse exécutive : Ce cours fondamental de R1.01 pose les bases indispensables de la programmation impérative et de l'architecture des données en BUT 1 Informatique. L'objectif est de comprendre comment un programme informatique interagit directement avec la mémoire vive (RAM) à travers le concept d'abstraction qu'est la variable (adresse mémoire, nom symbolique, type de donnée et valeur stockée). La séance démontre la nécessité d'un typage rigoureux pour contrer les dépassements de capacité (overflow), détaille les limites de précision de la représentation binaire en virgule flottante (norme IEEE 754), et explicite le pilotage du flux d'exécution séquentiel par des structures conditionnelles optimisées grâce au mécanisme d'évaluation paresseuse (short-circuit).

1. Le Concept de Variable et le Modèle Mémoire (RAM)

Pour exécuter un algorithme ou un programme C++, le processeur doit pouvoir lire et écrire temporairement des données en mémoire vive (RAM). Une variable est une abstraction matérielle haut niveau permettant au développeur d'interagir avec la RAM sans manipuler directement des adresses hexadécimales physiques. Une variable est définie par quatre caractéristiques indissociables : 1) Le Nom (ou identificateur), choisi par le développeur dans son code source ; 2) L'Adresse Mémoire, l'emplacement physique exact dans la barrette de RAM (ex: 0x7FFF5FBFF05C) ; 3) Le Type, qui indique au compilateur le nombre précis d'octets à réserver (ex: 4 octets pour un entier 'int') et la règle de décodage des bits ; 4) La Valeur, le motif binaire réellement inscrit dans cette case mémoire. Lors d'une affectation (ex: 'a = 5'), l'ordinateur se rend à l'adresse mémoire liée au nom 'a' et y enregistre le code binaire de 5. Si une nouvelle affectation survient ('a = 12'), l'ancienne valeur 5 est définitivement effacée et remplacée. La mémoire vive ne conserve aucun historique automatique des valeurs antérieures.

- Abstraction matérielle de la mémoire vive (RAM)
- Les quatre propriétés d'une variable : Nom, Adresse, Type, Valeur
- Affectation destructive (écrasement irréversible de l'ancienne valeur)

Cas concret enseignant : *Analogie du casier du vestiaire de l'IUT : Chaque casier possède un numéro d'emplacement fixe et immuable (l'adresse mémoire). Le développeur colle une étiquette sur la porte avec un nom symbolique (ex: 'age'). La taille physique du casier détermine son type et sa capacité d'accueil. Placer un sac rouge dans le casier équivaut à affecter la valeur 20. Si vous le remplacez ultérieurement par un sac bleu (valeur 21), le sac rouge disparaît définitivement du casier. Si vous tentez d'insérer un meuble trop volumineux dans le casier, il déborde ou casse (overflow).*

2. Types Primitifs, Représentation Binaire et Gestion du Dépassement (Overflow)

La sélection de la taille d'une variable borne la plage de valeurs manipulables. Un octet (8 bits) offre 2^8 = 256 combinaisons. Un entier signé sur 32 bits varie de -2 147 483 648 à +2 147 483 647. Lorsqu'un calcul ou une affectation dépasse cette limite maximale, le bit de poids fort bascule et la valeur 'boucle' brutalement vers la borne minimale négative : c'est le dépassement de capacité (overflow). Concernant les nombres à virgule, les systèmes informatiques appliquent la norme IEEE 754. Tout comme 1/3 ne possède pas de développement décimal fini en base 10 (0.3333...), des nombres simples comme 0.1 ou 0.2 n'ont pas de représentation binaire exacte en base 2. En conséquence, l'opération 0.1 + 0.2 produit 0.30000000000000004 dans l'ordinateur. Par conséquent, il est strictement proscrit de comparer deux flottants avec l'opérateur '==' en partiel. Pour valider l'égalité entre deux flottants 'x' et 'y', il faut s'assurer que la valeur absolue de leur différence est inférieure à un seuil d'erreur toléré (un epsilon, ex: std::abs(x - y) < 0.00001).

- Capacité des types entiers et phénomène d'Overflow (dépassement de capacité)
- Norme IEEE 754 et imprécision structurelle des nombres flottants
- Interdiction d'égalité stricte sur les flottants et méthode d'écart epsilon
- Règle de gestion financière : stockage sous forme d'entiers en centimes

Cas concret enseignant : *Catastrophe du vol 501 d'Ariane 5 (1996) : Le lanceur a été détruit 37 secondes après son décollage en raison d'une exception d'overflow. Un logiciel de bord a tenté de convertir un nombre flottant 64 bits en un entier signé 16 bits. La valeur ayant dépassé 32 767, le calculateur de bord a planté. Règle d'or de l'enseignant : 'N'utilisez jamais le type float pour stocker des prix ou des montants bancaires ! Représentez toujours les sommes financières par des entiers exprimés en centimes'.*

3. Structures Conditionnelles et Évaluation Paresseuse (Short-Circuit)

Par défaut, les instructions d'un programme s'exécutent de façon séquentielle, ligne après ligne de haut en bas. Les structures conditionnelles permettent de dévier ce flux selon la valeur de vérité (VRAI ou FAUX) d'une expression booléenne. L'évaluation paresseuse (ou court-circuit) est un mécanisme d'optimisation fondamental des opérateurs logiques : 1) Dans une expression '(A && B)', si 'A' est FAUX, l'ensemble de l'expression est obligatoirement FAUX ; l'ordinateur n'évalue même pas 'B'. 2) Dans une expression '(A || B)', si 'A' est VRAI, l'ensemble est obligatoirement VRAI ; l'ordinateur n'évalue pas 'B'. Outre l'optimisation des temps de calcul, l'évaluation paresseuse sécurise le code en empêchant des erreurs fatales d'exécution, telles que la tentative d'accès à une variable non initialisée ou une division par zéro.

- Exécution séquentielle vs Branchements conditionnels (SI / SINON SI / SINON)
- Opérateurs logiques ET (&&), OU (||), NON (!)
- Évaluation paresseuse (Short-circuit evaluation) et prévention de crashs

Cas concret enseignant : *Exemple d'un système d'authentification : Considérez l'instruction 'SI (utilisateur_existe ET mot_de_passe_correct)'. Si la fonction 'utilisateur_existe' renvoie FAUX, le système n'exécute jamais la requête de vérification du mot de passe en base de données. Cela économise des accès réseau et BDD coûteux et prévient un plantage critique du programme (de type NullPointer ou Segmentation Fault) si l'objet utilisateur n'est pas instancié en mémoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi 0.1 + 0.2 n'est-il pas exactement égal à 0.3 lorsqu'on l'affiche en programmation ?
Réponse du professeur : C'est une conséquence directe du codage des nombres flottants selon la norme IEEE 754. L'ordinateur stocke les nombres en base 2 (binaire). De même qu'il est impossible de représenter 1/3 sous forme décimale finie en base 10 (0.3333...), des fractions décimales simples comme 0.1 (1/10) ou 0.2 (2/10) n'ont pas de représentation binaire exacte finie en base 2. Le calcul 0.1 + 0.2 engendre un très léger biais d'arrondi binaire qui donne environ 0.30000000000000004. C'est pourquoi on ne compare JAMAIS deux flottants avec '==='.

Question étudiant : Est-ce qu'une variable déclarée à l'intérieur d'un bloc SI existe toujours après la fin du bloc SINON ?
Réponse du professeur : Non, absolument pas. Cela touche au concept de portée des variables (scope). Une variable déclarée dans un bloc délimité par des accolades `{ ... }` naît à sa déclaration et est physiquement détruite de la mémoire dès que l'exécution franchit l'accolade fermante `}` de ce bloc. Si vous tentez de l'utiliser en dehors du bloc SI ou SINON, le compilateur générera une erreur 'identifier not found'.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C++ qui demande l'âge d'un usager ainsi que son statut étudiant (booléen), puis calcule et affiche le tarif exact de son billet de cinéma selon la grille tarifaire suivante : - Tarif plein de base : 10.00 € - Tarif mineur (< 18 ans) : 6.00 € - Tarif étudiant majeur (>= 18 ans et statut étudiant VRAI) : 7.50 € - Tarif senior (>= 65 ans) : 5.00 € Le code doit scrupuleusement respecter les normes C++ de l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, commentaires de flot de données,

accolades Allman).

Démarche de résolution :
Étape 1 : Déclaration des constantes de prix et initialisation des variables d'entrée ('age', 'estEtudiant') et de sortie ('prixBillet'). Étape 2 : Lecture contrôlée des saisies clavier. Étape 3 : Ordonnancement rigoureux de l'enchaînement conditionnel du tarif le plus prioritaire/avantageux au cas général : - Le premier test 'if (age < 18)' traite tous les mineurs. Un mineur de 16 ans qui est étudiant bascule immédiatement ici et paiera 6.00 € (qui est le tarif le plus avantageux pour lui). - Le deuxième test 'else if (age >= 65)' traite les seniors. - Le troisième test 'else if (estEtudiant)' traite spécifiquement les majeurs étudiants qui paieront 7.50 €. - La branche 'else' finale attribue le tarif plein (10.00 €) par défaut.

```
/*
 * Programme : CalculTarifCinema
 * Finalité : Déterminer le tarif d'un billet de cinéma selon l'âge et le statut étudiant.
 * Auteur : Matéi de Saint Leger
 * Matière : R1.01 - Initiation au Développement
 */

#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES
    const float TARIF_PLEIN = 10.00f;
    const float TARIF_MINEUR = 6.00f;
    const float TARIF_ETUDIANT = 7.50f;
    const float TARIF_SENIOR = 5.00f;

    // VARIABLES
    int age;
    bool estEtudiant;
    float prixBillet;

    // TRAITEMENTS
    // (écran) << afficher message de saisie d age
    cout << "Entrez votre age : ";
    // (clavier) >> saisir >> age
    cin >> age;

    // (écran) << afficher message de saisie statut etudiant
    cout << "Etes-vous etudiant (1 pour Oui, 0 pour Non) : ";
    // (clavier) >> saisir >> estEtudiant
    cin >> estEtudiant;

    // Évaluation des tarifs dans l ordre des priorités tarifaires
    if (age < 18)
    {
        prixBillet = TARIF_MINEUR;
    }
    else if (age >= 65)
    {
        prixBillet = TARIF_SENIOR;
    }
    else if (estEtudiant)
    {
        prixBillet = TARIF_ETUDIANT;
    }
    else
    {
        prixBillet = TARIF_PLEIN;
    }

    // (écran) << afficher resultat final
    cout << "Le prix du billet est de : " << prixBillet << " euros." << endl;

    return 0;
}
```

Résultat : Validation des cas de test : 1) Usager de 20 ans, estEtudiant = VRAI -> '20 < 18' (FAUX), '20 >= 65' (FAUX), 'estEtudiant' (VRAI) -> Tarif attribué : 7.50 € (Conforme). 2) Usager de 16 ans, estEtudiant = VRAI -> '16 < 18' (VRAI) -> Tarif attribué : 6.00 € (L'ordre des conditions garantit l'application du tarif le plus bas).

■ Auto-Évaluation Flash d'Examen

Q : Que se produit-il en mémoire lors d'un dépassement de capacité (overflow) sur un entier signe sur 32 bits ?

R : La valeur binaire dépasse le nombre de bits alloués. Le bit de poids fort est tronqué ou inversé, ce qui provoque le basculement (ou bouclage) immédiat de la valeur vers la borne minimale négative (-2 147 483 648).

Q : Dans une condition 'if (A || B)', quel est l'intérêt de l'évaluation paresseuse (short-circuit) si l'expression A est évaluée à VRAI ?

R : Si A est VRAI, l'expression globale 'A || B' sera mathématiquement VRAIE quel que soit le résultat de B. L'ordinateur n'évalue donc pas l'expression B, ce qui gagne du temps de calcul et évite l'exécution d'instructions potentiellement dangereuses dans B.

Q : Quelle est la règle absolue à respecter lors de la comparaison de deux variables de type float en évaluation d'algorithmique ou C++ ?

R : Il est strictement interdit d'utiliser l'opérateur d'égalité stricte '=='. Il faut comparer la valeur absolue de leur différence à une valeur seuil epsilon très petite : 'std::abs(a - b) < epsilon'.